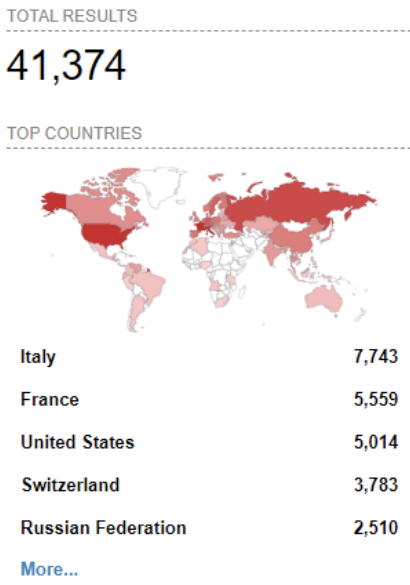


Posted on 3 May 2022 Maurizio Agazzini

Zyxel firmware extraction and password analysis

Backstory

During a **red teaming** exercise conducted for one of our customers, we abused weak passwords to obtain an administrative access to some **Zyxel ZyWALL Unified Security Gateway (USG)** appliances that were used as both firewalls and VPN concentrators in their branch offices. These appliances are targeted at small and medium businesses and are somewhat popular, at least according to **Shodan**.



Thanks to our administrative access, we were able to dump the configurations and we noticed that a series of passwords stored on these devices were encrypted in some way. When we did a Google search we could not find any public information on the Internet about how these passwords were stored. Based on our observations, they had to be encrypted with a **reversible algorithm** because **they were passwords that the device itself used**, such as the PSKs of VPNs.

```
! model: USG310
! firmware version: 4.05(AAPJ.1)
!
description lan
!
hardware-watchdog-timer start
!
software-watchdog-timer 60
!
interface-name ge1
interface-name ge2
interface-name ge3
interface-name ge4
interface-name ge5
interface-name ge6
interface-name ge7
interface-name ge8
!
led_suppress 60 [REDACTED] disable
!
led_locator 50 [REDACTED] blink-timer 10
!
aaa group server radius radius
server encrypted-key $[REDACTED]$
server acct-interin-interval 10
server acct-retry-count 3
server group-attribute 11
server timeout 5
server host 192.168 [REDACTED] auth-port 1812
!
```

Since we had some spare budget, we decided to buy a **similar device** on eBay and spend some time auditing it on our own. Our next articles will cover the results of our analysis carried out on the physical device. In the meantime, let’s focus on the work we did **while we were waiting for the device to arrive**.

In this first article of our **Zyxel audit series** we will cover **firmware extraction** and **password decryption** against Zyxel ZyWALL Unified Security Gateway (USG) appliances.

Firmware extraction

First of all, we downloaded from the **official website** the same firmware images (version 4.10 and 4.70) for the USG310 device that were deployed by our customer.

We started working on version 4.10 of the firmware: the file 410AAPJ2C0.bin is the firmware image in ZIP format. Unfortunately, the ZIP archive was **password protected**. Searching online we were unable to find any information on the passwords used by Zyxel.

In the PDF documents distributed with the firmware, however, a recovery method is described in the event that the upgrade procedure is unsuccessful.

BLOG CATEGORIES

- ARTICLES
- EVENTS
- EXPLOITS
- NEWS
- TOOLS
- VULNERABILITIES

TAGS

0day advisory ARM
bindiff brida Burp Suite
Bus Pirate BUSSide
c/c++ code review cve-
2021-22205 CVE-2022-
0342 CVE-2022-2030
cve-2022-26531 cve-
2022-26532
disassembly dynamic
analysis EL EL Injection
exploit firmware format
string frida ghidra
ghidra2frida gitlab
golang groovy Hack In
Paris hacking HackRF
hardware hn security I2C
incident response
infiltrate iOS iot Java
Java Deserialization
Scanner Java
Serialization Keil log4j
metasploit meterpreter
mips mobile OpenSSH
penetration test php
Phrack pseudocode
radio RCE research
reverse engineering
romhack Security
Manager semgrep
shellcode solaris source
code analysis sparc
static analysis tactical
exploitation Template
Injection Universal Radio
Hacker vulnerability
research windows x86 yii
zyxel



```
● Press any key to enter debug mode
U-Boot 2011.03 (Development build, svnversion: u-boot:422M, exec:exported)
(Build time: Feb 21 2013 - 10:15:57)

BootModule Version: V1.07 | 02/21/2013 10:45:46
DRAM: Size = 2048 Mbytes

Press any key to enter debug mode within 3 seconds.
.....

EnterDebug Mode
ZW1100>

● Enter atkz -f -l 192.168.1.1 to configure FTP server IP address
>
>
>
> atkz -f -l 192.168.1.1

● Enter atgof to bring up the FTP server on port 1
ZyWALL 1100> atgof

Booting...

● The following information shows the FTP service is up and ready to receive FW
Building ...

Connect a computer to port 1 and FTP to 192.168.1.1 to upload the new file.
[

● You will use FTP to upload the firmware package. Keep the console session open in order
to see when the firmware update finishes.
```

A lower level procedure is also available in case the previously described method does not work:

<https://kb.zyxel.com/KB/searchArticle!gwsViewDetail.action?articleOid=006845&lang=EN>

Based on the information in the low level recovery guide we can assume that the “.ri” file is directly executed by the system to install the fundamental components in order to then be able to flash the firmware from the “.bin” file.

We extracted “410AAPJ2C0.ri” with binwalk:

```
inode@stormbringer:w$ binwalk -e 410AAPJ2C0.ri

DECIMAL          HEXADECIMAL      DESCRIPTION
-----
512               0x200            uImage header, header size: 64 bytes, header CRC: 0xF9EC01
576               0x240            LZMA compressed data, properties: 0x5D, dictionary size: 8

inode@stormbringer:w$
```

And the files inside it:

```
inode@stormbringer:w/_410AAPJ2C0.ri.extracted$ binwalk -e 240

DECIMAL          HEXADECIMAL      DESCRIPTION
-----
0                0x0              ELF, 64-bit MSB MIPS32 rel2 executable, MIPS, version 1 (S
5095576          0x4DC098         Linux kernel version 2.6.32
11385704         0xADBB68         gzip compressed data, maximum compression, from Unix, last
11487616         0xAF4980         DES SP2, big endian
11488128         0xAF4B80         DES SP1, big endian
11511376         0xAFA650         CRC32 polynomial table, little endian
11910064         0xB5BBB0         Unix path: /usr/bin/magic-seed
11919441         0xB5E051         Unix path: /sys/module/perf_counters/parameters/counter{0,
11919495         0xB5E087         Unix path: /sys/module/perf_counters/parameters/l2counter{
11948723         0xB652B3         PARity archive data - file number 16717
12987688         0xC62D28         Unix path: /home/sddl/source/410/formal/p2/usg310/src/kern
13224040         0xC9C868         Neighborly text, "NeighborSolicits"
13224064         0xC9C880         Neighborly text, "NeighborAdvertisementsorts"
13227391         0xC9D57F         Neighborly text, "neighbor %.2x%.2x%.2x%.2x%.2x%.2x%.2x%.
13231704         0xC9E658         Flattened device tree, size: 10827 bytes, version: 17
13242536         0xCA10A8         Flattened device tree, size: 11728 bytes, version: 17
13622896         0xCFDE70         Unix path: /usr/local/zld_udev/sbin/uevent_helper.sh
14045184         0xD65000         gzip compressed data, maximum compression, from Unix, last
15042374         0xE58746         Zip archive data, at least v2.0 to extract, compressed siz
15269496         0xE8FE78         Zip archive data, at least v2.0 to extract, compressed siz
15880177         0xF24FF1         Zip archive data, at least v2.0 to extract, compressed siz

inode@stormbringer:w/_410AAPJ2C0.ri.extracted$
```

And again:

```
inode@stormbringer:w/_410AAPJ2C0.ri.extracted/_240.extracted$ binwalk -e D65000

DECIMAL          HEXADECIMAL      DESCRIPTION
-----
0                0x0              ASCII cpio archive (SVR4 with no CRC), file name: ".", fil
112              0x70             ASCII cpio archive (SVR4 with no CRC), file name: "zyinit"
232              0xE8             ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
266064           0x40F50          ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
269196           0x41B8C          ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
709204           0xAD254          ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
715356           0xAEA5C          ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
2879904          0x2BF1A0         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3151288          0x3015B8         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3196236          0x30C54C         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3290728          0x323668         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3312388          0x328B04         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3391488          0x33C000         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3396772          0x33D4A4         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3745356          0x39264C         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3745816          0x392818         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3768480          0x3980A0         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3946956          0x3C39CC         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3947164          0x3C3A9C         ASCII cpio archive (SVR4 with no CRC), file name: "zyinit/"
3966960          0x3C87F0         ASCII cpio archive (SVR4 with no CRC), file name: "init",
3967092          0x3C8874         ASCII cpio archive (SVR4 with no CRC), file name: "TRAILER

inode@stormbringer:w/_410AAPJ2C0.ri.extracted/_240.extracted$
```

Looking at the extracted files, it is possible to identify the “zyinit” binary, which contains some firmware related strings.

By analyzing it, it is possible to see that it launches other external commands, in particular the “zld_fsextract” command:

```
if (iStack68 != -1) {
    if (iStack68 == 0) {
        zld_fsextract();
    }
}
```



Searching online for these executable binaries, only an interesting URL was identified (<https://www.dslreports.com/forum/remark,26961186>), which gave us some information about the ZIP password:



Brano
I hate Vogons
MVM
Join 2002-06-25
Burlington, ON
 TekSavvy Cable
(Software) OPNsense
Ubiquiti UniFi UAP-AC-PRO
Ubiquiti NanoBeam M5 16

Seems to be

quote:

3.00(AQU.0) / 2012-02-17 16:50:38

You can always compare the .bin files once it shows on US ftp servers.

I would not worry that this is some kind of fake, USG firmwares are zipped with very strong password. The zld_fsextract update command then verifies the zip and unzips is before applying the update. For somebody to fake the firmware they must know how to generate the proper password (each update has different password) ... just watch console output during update if you're curious.

 +1 · actions · 2012-Mar-2 9:10 pm · [reply](#)

Searching in the “zld_fsextract” binary for the password, it is possible to identify some good starting points for the analysis, like this one:

```
if (DAT_10114fc0 == 0) {
    apcStack20812[0] = "-o";
    apcStack20812[1] = "-q";
    apcStack20812[2] = "-P";
    iStack112 = 6;
    pcVar9 = "-o";
    pcVar4 = "-q";
    pcVar5 = "-P";
    apcStack20812[3] = (char *)suStack816;
    uStack20796 = DAT_10114fcc;
    if (DAT_10114fc4 != (char *)0x0) {
        iStack112 = 8;
        puStack20792 = &DAT_100112c8;
        pcVar9 = apcStack20812[0];
        pcVar4 = apcStack20812[1];
        pcVar5 = apcStack20812[2];
        pcStack20788 = DAT_10114fc4;
    }
}
```

These options are used by the “unzip” binary to unzip a file with a specific password which is defined in the parameter “-P”. Based on information found online and on a quick analysis, it seems that the binary calculates the **unzip password** in some way based on the binary name or the binary content.

The fastest way to extract the files is to emulate the MIPS processor and execute the binary.

Launching a Linux/MIPS virtual machine:

```
> wget https://people.debian.org/~aurel32/qemu/mips/vmlinux-3.2.0-4-5kc-malta
> wget https://people.debian.org/~aurel32/qemu/mips/debian_wheezy_mips_standard.qcow2
> qemu-system-mips64.exe -M malta -kernel vmlinux-3.2.0-4-5kc-malta -hda debian_wheezy_m
```

Getting firmware image information using the “zld_fsextract” binary:

```
root@debian-mips:~# ./zld_fsextract 410AAPJ2C0.bin -s list
name          :kernel
scope         :--f kernelusg310.bin -f kernelchecksum -D /
nc_scope      :--f kernelusg310.bin
version       :2.6.32
build_date    :2014-12-06 11:27:35
checksum      :6e4e1ad212be0a8a3ce89484f3c5dc1e
core_checksum :a028057f7c742ea52bd3d0408f38a673

name          :code
scope         :--f bmusg310.bin -f bmchecksum -f kernelusg310.bin -f kernelchecksum
scope         :--d db/etc/zyxel/ftp/conf -D /
nc_scope      :--f fwversion -f filechecksum -f wtpinfo
version       :4.10 (AAPJ.2)
build_date    :2014-12-09 08:51:18
checksum      :899be95dac4a5bdcfd2f694035f16746
core_checksum :e25ab639d4ff432bb7ffdc8c1bd39be3

name          :WTP_wtp_image/400AAS4C0.bin
scope         :--f wtp_image/400AAS4C0.bin -D /db
nc_scope      :
version       :4.00(###.4)
build_date    :2013-08-31 20:36:43
checksum      :07773b3deb39a1dd9c2036201097529c
core_checksum :07773b3deb39a1dd9c2036201097529c

name          :WTP_wtp_image/400AADG4C0.bin
scope         :--f wtp_image/400AADG4C0.bin -D /db
nc_scope      :
version       :V4.00(###.4)
build_date    :2013-08-31 20:35:40
checksum      :aabb0606887cec28a07b8598e699f027
core_checksum :aabb0606887cec28a07b8598e699f027

root@debian-mips:~#
```

And using the zld_fsextract binary to extract the firmware without the need to specify a password:

```
root@debian-mips:~# ./zld_fsextract 410AAPJ2C0.bin ./unzip -s extract -e code
...
root@debian-mips:~#

root@debian-mips:~# ls -al /rw/
total 57192
drwxr-xr-x  3 root root    4096 Nov 23 12:56 .
drwxr-xr-x 24 root root    4096 Nov 23 12:56 ..
-r--r--r--  1 root root 58511360 Dec  6 2014 compress.img
drwxr-xr-x  5 root root    4096 Dec  9 2014 etc_writable
-rw-r--r--  1 root root    139 Dec  9 2014 filechecksum
-rw-r--r--  1 root root   21415 Dec  9 2014 filelist
-rw-r--r--  1 root root    326 Dec  9 2014 fwversion
-rw-r--r--  1 root root    1671 Dec  9 2014 wtpinfo
root@debian-mips:~#
```

Then uncompressing the new image:

```
inode@stormbringer:w$ binwalk -e compress.img
```

DECIMAL	HEXADECIMAL	DESCRIPTION
---------	-------------	-------------


```
inode@stormbringer:w$
```

After seeing that the process worked correctly with a fully emulated environment, we decided to try the single binary emulation provided by qemu. By using “strace” we can quickly see how the “unzip” binary was launched to find the ZIP password.

```
1. strace -f -s 199 qemu-mipsn32-static ./zld_fsextract 410AAPJ2C0.bin ./unzip -s ext
```

Here is the output.

```
D[CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID <unfinished ... >
[pid 2299] wait4(-1, strace: Process 2304 attached
<unfinished ... >
[pid 2304] set_robust_list(0x7f06600469e0, 24) = 0
[pid 2304] clock_nanosleep(CLOCK_REALTIME, 0, {tv_sec=0, tv_nsec=100000000}, <unfinished ... >
[pid 2303] <... clone resumed>, parent_tid=[2304], tls=0x7f0660046700, child_tidptr=0x7f06600469d0) = 2304
[pid 2303] rt_sigprocmask(SIG_SETMASK, ~[KILL STOP RTMIN RT_1], NULL, 8) = 0
[pid 2303] rt_sigprocmask(SIG_SETMASK, ~[RTMIN RT_1], NULL, 8) = 0
[pid 2303] rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
[pid 2303] execve("./unzip", ["/unzip", "-o", "-q", "-P", "IVq",
Xd", "410AAPJ2C0.bin", "-d", "/", "db/etc/zyxel/ftp/conf/", "db/etc/zyxel/ftp/conf/system-de
fault.conf"], 0x29dee00 /* 55 vars */) = -1 ENOENT (No such file or directory)
[pid 2303] write(2, "_exec_: ", 10_exec_: ) = 10
[pid 2303] write(2, ":", 2:) = 2
[pid 2303] write(2, "No such file or directory", 25No such file or directory) = 25
[pid 2303] write(2, "\n", 1
)
= 1
[pid 2303] rt_sigprocmask(SIG_SETMASK, ~[RTMIN RT_1], NULL, 8) = 0
[pid 2303] exit_group(0) = ?
[pid 2304] <... clock_nanosleep resumed> <unfinished ... >) = ?
[pid 2304] +++ exited with 0 +++
[pid 2303] +++ exited with 0 +++
[pid 2299] <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 2303
[pid 2299] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=2303, si_uid=1000, si_status=0, si_ut
ime=0, si_time=0} ---
= 1
[pid 2299] write(1, " ", 1.) = 1
[pid 2299] access("/etc/qemu-binfmt/mipsn32/db/etc/zyxel/ftp/conf/system-default.conf", F_OK) = -1 ENOENT
(No such file or directory)
[pid 2299] openat(AT_FDCWD, "//db/etc/zyxel/ftp/conf/system-default.conf", O_RDONLY) = -1 ENOENT (No such
file or directory)
[pid 2299] write(1, "\r\n", 2
)
= 2
[pid 2299] rt_sigprocmask(SIG_SETMASK, ~[RTMIN RT_1], NULL, 8) = 0
[pid 2299] exit_group(-66) = ?
[pid 2300] <... futex resumed>
= ?
[pid 2300] +++ exited with 190 +++
+++ exited with 190 +++
```

The binary that calculates the ZIP password is **statically compiled**, therefore it is not so easy to understand how that password is generated, and above all we are lazy...

Password encryption algorithm

Now that we have access to the filesystem, let’s start looking for information on the algorithm used to store passwords in the configuration files. In the configurations available to us, all passwords used the **\$4\$** format and were preceded by the following keywords:

- encrypted-key
- encrypted-keystring
- encrypted-password
- encrypted-presharekey
- password-encrypted
- wpa-psk-encrypted

First step, identify which executables deal with these types of keywords:

```
inode@stormbringer:w$ grep "encrypted-" _compress.img.extracted/squashfs-root/bin/*
Binary file _compress.img.extracted/squashfs-root/bin/zysh matches
Binary file _compress.img.extracted/squashfs-root/bin/zyshd matches
inode@stormbringer:w$
```

Looking in the code of “zyshd” for the string “\$4\$” which identifies the encrypted password:

```
int match_original_ZLD_pwd(char *ori_pwd, char *clear_pwd)
{
    int iVar1;
    char new_pwd [129];
    char buffer [129];
    char salt [9];

    strncpy(salt, ori_pwd + 3, 8);
    salt[8] = '\0';
    .pic.schrodinger(clear_pwd, buffer, salt);
    sprintf(new_pwd, "%4$%s$%s", salt, buffer);
    iVar1 = strcmp(ori_pwd, new_pwd, 0x80);
    return (int) (uint) (iVar1 != 0);
}
```

Looking inside the schrodinger routine. This part generates the salt:

```
iVar16 = open("/dev/urandom", 0);
if (iVar16 == -1) goto LAB_10300d04;
pbStack108 = abStack136;
iVar22 = 1;
read(iVar16, pbStack108, 0x10);
iVar17 = 1;
*(char *)puVar21 =
".ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789abcdefghijklmnopqrstuvwxyz"[abStack136[0] & 0x3f];
do {
    *(char *) (iVar22 + (int)puVar21) =
".ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789abcdefghijklmnopqrstuvwxyz"
[abStack136[iVar17] & 0x3f];
    if (iVar17 == 0xf) {
        read(iVar16, pbStack108, 0x10);
        iVar17 = 1;
    }
    else {
        iVar17 = iVar17 + 1;
    }
    iVar22 = iVar22 + 1;
} while (iVar22 != 8);
*(byte *) (puVar21 + 1) = 0;
close(iVar16);
```

Encryption is performed by the following code:

```
uStack144 = 0xffffffffffffffff;
uStack152 = CONCAT71(CONCAT61(CONCAT51(CONCAT41(CONCAT31(CONCAT21(CONCAT11(euclid._0_1_,
                                                    euclid._1_1_),
                                                    euclid._2_1_),euclid._3_1_
                                                    euclid._4_1_),euclid._5_1_),euclid._6_1_),
                                                    (undefined)euclid);
pvVar14 = malloc(sStack156 * 2 + 0x124);
uVar12 = (undefined)euclid;
uVar11 = euclid._6_1_;
uVar10 = euclid._5_1_;
uVar9 = euclid._4_1_;
uVar4 = euclid._3_1_;
uVar3 = euclid._2_1_;
cVar8 = euclid._1_1_;
uVar2 = euclid._0_1_;
if (pvVar14 != (void *)0x0) {
    iVar15 = (int)pvVar14 + uStack112;
    cVar19 = '\x06' - euclid._1_1_;
    *(undefined *) (iVar15 + 0xe) = 10;
    *(undefined *) (iVar15 + 10) = 0x23;
    *(undefined *) (iVar15 + 0x17) = 0x13;
    *(undefined *) (iVar15 + 0x13) = 0x8f;
    *(undefined *) (iVar15 + 7) = 5;
    *(undefined *) (iVar15 + 0xb) = 0xfb;
    *(undefined *) (iVar15 + 9) = 0x1f;
    *(undefined *) (iVar15 + 8) = 0x4a;
    *(undefined *) (iVar15 + 0xc) = uVar11;
    *(undefined *) (iVar15 + 0x10) = 0xcd;
    *(undefined *) (iVar15 + 0x14) = 0x5a;
    *(undefined *) (iVar15 + 4) = uVar3;
    *(undefined *) (iVar15 + 6) = uVar2;
    *(undefined *) (iVar15 + 0xf) = 0x14;
    *(undefined *) (iVar15 + 0x11) = 0xd;
    *(undefined *) (iVar15 + 0x12) = uVar9;
    *(undefined *) (iVar15 + 0x15) = uVar12;
    *(char *) (iVar15 + 0x16) = cVar19;
    *(undefined *) (iVar15 + 0x18) = 6;
    *(undefined *) (iVar15 + 0x19) = uVar10;
    *(undefined *) (iVar15 + 0x1a) = 0x12;
    *(undefined *) (iVar15 + 0x1b) = uVar4;
    *(undefined *) (iVar15 + 5) = 0x12;
    *(char *) (iVar15 + 0xd) = cVar8;
    RIJ_set_encrypt_key(iVar15 + 4,0xc0,iVar15 + 0x1c);
    if (*(char *)param_3 == '\0') {
        *param_3 = puVar20;
    }
    RIJ_cbc_encrypt(puVar20,puVar20,0x50,iVar15 + 0x1c,&uStack152,1);
    free(pvVar14);
}
```

And then the BASE64 encoding at the end:

```
do {
    bVar1 = *(byte *)puVar18;
    pbVar5 = (byte *) ((int)puVar18 + 1);
    pbVar6 = (byte *) ((int)puVar18 + 2);
    cVar2 = "ABCDEFGHGIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
        [(bVar1 & 3) * 0x10 + (uint) (*pbVar5 >> 4)];
    puVar18 = (undefined8 *) ((int)puVar18 + 3);
    cVar20 = "ABCDEFGHGIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
        [(pbVar5 & 0xf) * 4 + (uint) (*pbVar6 >> 6)];
    cVar3 = "ABCDEFGHGIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"[*pbVar6 & 0x3f];
    *pcVar19 = "ABCDEFGHGIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"[bVar1 >> 2];
    pcVar19[1] = cVar2;
    pcVar19[2] = cVar20;
    pcVar19[3] = cVar3;
    pcVar19 = pcVar19 + 4;
} while (puVar18 != (undefined8 *) ((int)puVar21 + 0x4e));
cVar2 = "ABCDEFGHGIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    [(byte *) ((int)puVar21 + 0x4e) >> 2];
cVar20 = "ABCDEFGHGIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    [(byte *) ((int)puVar21 + 0x4e) & 3) * 0x10 +
    (uint) (* (byte *) ((int)puVar21 + 0x4f) >> 4)];
cVar3 = "ABCDEFGHGIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    [(byte *) ((int)puVar21 + 0x4f) & 0xf) * 4];
param_2[0x6b] = '$';
param_2[0x68] = cVar2;
param_2[0x69] = cVar20;
param_2[0x6a] = cVar3;
param_2[0x6c] = '\0';
free(__ptr);
return 0;
```

Looking inside the “RIJ_cbc_encrypt” function, it is possible to identify the routine “RIJ_decrypt” which leads us to the **AES algorithm**. This is also confirmed by the name of the function (RIJ = Rijndael) and by the 0xC0 used on the set key, which is 192. AES is one of the few encryption algorithms that support 192-bit keys.

Looking inside the encryption routine, it is possible to identify AES parameters like **Td0** memory; it is also possible to use the **FindCrypt** Ghidra plugin that helps identify the encryption routines in use.

We can assume that the **encryption/decryption** function works as follows:

```
RIJ_cbc_encrypt(destination, source, len, unknown, IV, encryption/decryption byte)
```

At this point we only need to identify the parameters passed to the function.

The IV is:

```
uStack144 = 0xffffffffffffffff;
uStack152 = CONCAT71(CONCAT61(CONCAT51(CONCAT41(CONCAT31(CONCAT21(CONCAT11(euclid._0_1_,
                                                    euclid._1_1_),
                                                    euclid._2_1_),euclid._3_1_
                                                    euclid._4_1_),euclid._5_1_),euclid._6_1_),
                                                    (undefined)euclid);
```

The encryption key is:


```
*(undefined *) (iVar16 + 0xe) = 10;
*(undefined *) (iVar16 + 10) = 0x23;
*(undefined *) (iVar16 + 0x17) = 0x13;
*(undefined *) (iVar16 + 0x13) = 0x8f;
*(undefined *) (iVar16 + 7) = 5;
*(undefined *) (iVar16 + 0xb) = 0xfb;
*(undefined *) (iVar16 + 9) = 0x1f;
*(undefined *) (iVar16 + 8) = 0x4a;
*(undefined *) (iVar16 + 0xc) = uVar12;
*(undefined *) (iVar16 + 0x10) = 0xcd;
*(undefined *) (iVar16 + 0x14) = 0x5a;
*(undefined *) (iVar16 + 4) = uVar8;
*(undefined *) (iVar16 + 6) = uVar7;
*(undefined *) (iVar16 + 0xf) = 0x14;
*(undefined *) (iVar16 + 0x11) = 0xd;
*(undefined *) (iVar16 + 0x12) = uVar10;
*(undefined *) (iVar16 + 0x15) = uVar13;
*(char *) (iVar16 + 0x16) = cVar20;
*(undefined *) (iVar16 + 0x18) = 6;
*(undefined *) (iVar16 + 0x19) = uVar11;
*(undefined *) (iVar16 + 0x1a) = 0x12;
*(undefined *) (iVar16 + 0x1b) = uVar9;
*(undefined *) (iVar16 + 5) = 0x12;
*(char *) (iVar16 + 0xd) = cVar2;
```

So:

```
aes_key = "001200054A1F23FB1F060A14CD0D018F5AC0001306F0121C"
aes_iv = "0006001C01F0FC0FFFFFFFFFFFFFFFF"
```

The same process can be followed on the firmware version 4.70 to identify the \$5\$ encryption scheme.

To summarize, in the case of a password hash starting with \$4\$ the algorithm is as follows:

```
1. iv = static_iv
2. key = static_key
3. salt = generate_8_bytes_from_random()
4. to_encrypt = salt + password (the password is repeated to reach the 80 bytes)
5. AES_SET_KEY(aes, key, 192)
6. AES_DECRYPT(aes, cyphertext, to_encrypt, key, iv)
7. final_encrypted = base64(cyphertext)
8. final_string = "$4$" + salt + "$" + final_encrypted "$"
```

In the case of a password hash starting with \$5\$ the algorithm is as follows:

```
1. iv = static_iv
2. key = static_key
3.
4. salt = generate_8_bytes_from_random()
5. to_encrypt = salt + password (the password is repeated to reach the 80 bytes)
6. AES_SET_KEY(aes, key, 192)
7. AES_DECRYPT(aes, cyphertext, to_encrypt, key, iv)
8.
9. salt = generate_8_bytes_from_random()
10. to_encrypt = salt + password (the password is repeated to reach the 80 bytes)
11. AES_SET_KEY(aes, key, 192)
12. AES_DECRYPT(aes, cyphertext, to_encrypt, key, iv)
13. step1 = base64(cyphertext)
14.
15. salt1 = generate_8_bytes_from_random()
16. to_encrypt = salt1 + step1
17. AES_SET_KEY(aes, key, 192)
18. AES_DECRYPT(aes, cyphertext, to_encrypt, key, iv)
19. final_encrypted = base64(cyphertext)
20.
21. final_string = "$5$" + salt1 + "$" + salt2 + "$" + final_encrypted "$"
```

We developed a decryption tool that can be downloaded from: https://github.com/inode-/zyxel_password_decrypter

Another possible way to reverse these algorithms is to use dynamic analysis, but it seems QEMU does not fully support the SoC used by Zyxel and generates some illegal instruction fault on most executables. There is a potentially working QEMU patch at <https://github.com/amir-mehmood/QEMU-Octeon-MIPS64>, but we did not test it.

Finally, when we received the physical device we found out that the /usr/sbin/zencrypt binary present in the filesystem can be used to decrypt these passwords as well:

```
root@USG20~VPN:~# zencrypt -k 'E8AzAwOyYBwVHJJtUTDqdAxLDPIr/ncDNV5HCf2FRJlqhYkSqbGvX9BP'
pluto
```

Stay tuned for our upcoming articles in this series, along with some juicy vulnerability disclosures!